

Parte C: preguntas de repaso e investigación

1) ¿Por qué SJF no es implementable tal cual en un SO real y cómo se aproxima?

SJF (Shortest Job First) requiere conocer de antemano la duración exacta de la ráfaga de CPU de cada proceso. En un SO real esto es imposible, ya que el tiempo de ejecución depende de datos, ramas, E/S imprevista o comportamiento dinámico.

****Cómo se aproxima:****

- ****SRTF (Shortest Remaining Time First):**** versión preventiva que usa el tiempo restante estimado.
- ****Estimación por ráfagas pasadas:**** se usa **exponential averaging** para predecir el siguiente burst.
- ****MLFQ (Multilevel Feedback Queue):**** ajusta prioridades según comportamiento observado.
- ****CFS (Completely Fair Scheduler):**** no usa burst conocido, busca equidad basada en tiempo virtual.

2) Explicar el efecto convoy y qué algoritmo lo evita

El ****efecto convoy**** ocurre con FCFS cuando un proceso largo monopoliza la CPU. Los procesos cortos que llegan después deben esperar hasta que termine, incluso si solo necesitan poco tiempo, lo que retrasa E/S y reduce eficiencia.

****Algoritmos que lo evitan:****

- ****Round Robin (RR):**** reparte CPU por quantum, impide monopolización.
- ****SRTF:**** prioriza procesos con menor tiempo restante.
- ****MLFQ:**** da preferencia a procesos cortos/interactivos.
- ****CFS:**** garantiza reparto justo entre procesos listos.

3) ¿Qué es el envejecimiento y qué problema resuelve?

El **envejecimiento (*aging*)** es una técnica que incrementa gradualmente la prioridad de un proceso conforme más tiempo lleva esperando en la cola de listos.

Problema que resuelve: la **inanición (*starvation*)**. En planificadores con prioridades fijas, los procesos de baja prioridad pueden nunca ejecutarse si llegan constantemente procesos de mayor prioridad. Con envejecimiento, tras un tiempo suficiente, un proceso de baja prioridad alcanza prioridad suficiente para obtener CPU.

4) Investigar cómo planifica procesos el kernel de Linux actual (CFS o su sucesor) y comparar con Round Robin con multicolos

CFS (Completely Fair Scheduler):

- Introducido en Linux 2.6.23. En núcleos modernos se mantiene CFS con extensiones (EEVDF propuesto como sucesor, pero CFS sigue muy extendido).
- Usa **virtual runtime (vruntime)**: tiempo de CPU normalizado según peso (*nice*).
- Mantiene procesos listos en un **árbol rojo-negro** ordenado por vruntime; elige siempre el nodo con menor vruntime.
- **Preventivo:** si llega un proceso con vruntime menor, se conmuta.
- **Sleeper fairness:** premia a procesos que volvieron de E/S.
- **Multicore:** balanceo de carga entre CPUs, afinidad, dominios de planificación.

Comparación con Round Robin con multicolos:

Aspecto	CFS	RR con multicolos
Criterio	Equidad proporcional (nice)	Rotación por quantum fijo
Selección	Menor vruntime (árbol RB)	Siguiente en cola circular
Quantum	Dinámico/implícito	Fijo por núcleo
Multicore	Balanceo inteligente + afinidad	Por cola por núcleo (simple)
Interactivos	Muy buenos (sleeper fairness)	Menos adaptado
Complejidad	Media-alta	Baja
Objetivo	Máxima equidad	Rotación equitativa

5) Reimplementar FCFS de la sección 11 en el otro lenguaje (Java a C) sin copiar el código: transcribirlo a mano y explicar cada línea

Código reimplementado en C (FcFs_C_reimplementacion.c):

```
#include <stdio.h>

int main() {
    int n, i, j;
    int at[10], bt[10], wt[10], tat[10];
    float avgWt = 0, avgTat = 0;

    printf("Ingrese el numero de procesos: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Proceso %d - Tiempo de llegada: ", i + 1);
        scanf("%d", &at[i]);
        printf("Proceso %d - Tiempo de ráfaga (burst): ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = 0;
        for (j = 0; j < i; j++) {
            wt[i] += bt[j];
        }
        wt[i] -= at[i];
        if (wt[i] < 0) wt[i] = 0;
    }

    printf("\nProceso\tLlegada\tBurst\tEspera\tTAT\n");
    for (i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        avgWt += wt[i];
        avgTat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
    }

    avgWt /= n;
    avgTat /= n;

    printf("\nPromedio Tiempo de Espera = %.2f", avgWt);
    printf("\nPromedio Turnaround Time = %.2f\n", avgTat);

    return 0;
}
```

****Explicación línea por línea:****

- #include <stdio.h> – Entrada/salida estándar.
- int main() { ... } – Función principal.
- int n,i,j; – Variables para contar.
- int at[10], bt[10], wt[10], tat[10]; – Arreglos llegada, burst, espera, TAT.
-
- loat avgWt=0, avgTat=0; – Acumuladores promedios.
- printf/scanf – Solicita número de procesos.
- Bucle i=0..n-1: ingresa AT y BT.
- wt[0]=0 – Primer proceso no espera.
- Bucle i>=1: wt[i] = suma(bt[0]..bt[i-1]) - at[i]. Si < 0 → 0.
- Segundo bucle: calcula at[i]=bt[i]+wt[i], acumula y muestra tabla.
- Divide por n para promedios.
-
- eturn 0; – Fin.